

ICML 2024 | 让半定规划真正跑得动：USBS 把百亿变量的 SDP 变成可用工具

UMass Amherst 提出统一谱束方法 USBS，支持热启动、可扩展到 10^{13} 级决策变量，比前一代求解器快约 500×

半定规划（Semidefinite Programming，SDP）几乎是组合优化里最“万能”的工具之一：从 MaxCut、二次分配到神经网络验证与控制，很多难题都能松弛成一个 SDP 来求解。但一个长期的共识是——它太贵了。经典求解器每一步都要把矩阵投影回半定锥，这需要一次完整的特征分解，计算量随问题规模呈立方增长。于是在实践中，SDP 常被贴上“理论优美、规模上不可行”的标签。

近年的草图化（sketching）方法，比如 CGAL，用低秩草图代替存储完整矩阵，把可解规模往前推了一大步。但它们也付出了代价：问题越大，需要的迭代次数越多，收敛越慢；更麻烦的是，这类方法依赖“随迭代变化的参数表”，这恰好挡住了热启动（warm-start）——也就是用上一次的解来初始化下一个相近问题的能力。而在数据增量到达、或在混合整数 / 交互式循环里反复求解一串相近 SDP 的场景中，热启动几乎是提速的关键。

ICML 2024 的这篇工作提出 USBS（Unified Spectral Bundling with Sketching，统一谱束加草图方法）：一个可证明收敛的求解器，它同时处理等式与不等式约束，能可靠地利用热启动，并在可选的矩阵草图下扩展到极大规模。在一个超过 2×10^9 个决策变量的 MaxCut 实例上，USBS 比前一代最强的可扩展求解器快约 500×；而单是热启动本身，就能把收敛再加速 100× 以上。

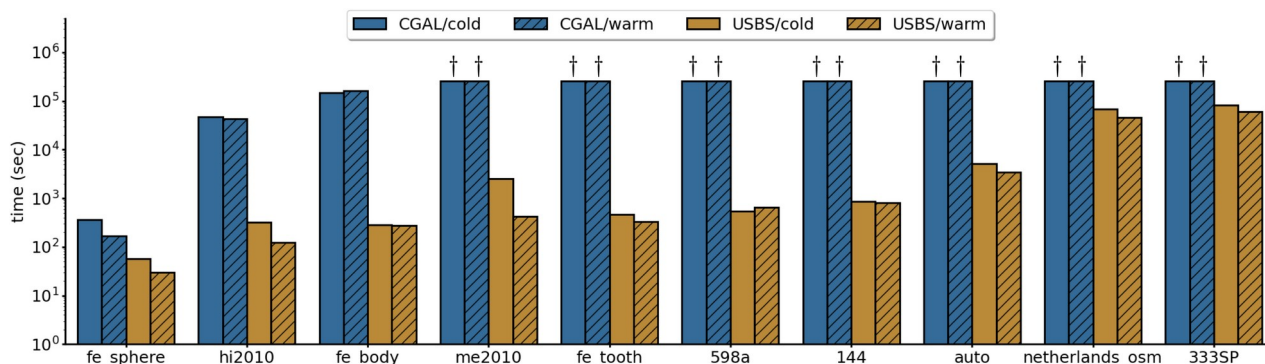


图 1 收敛到中等精度 ($\epsilon=10^{-1}$) 所需时间 (秒，越低越好)。在十个 DIMACS10 MaxCut 实例上比较 CGAL 与 USBS 的冷启动 / 热启动 (用 99% 数据构造热启动)，实例按顶点数 n 从 16K 到 3.7M 升序排列。带 $\dagger$ 的柱表示 72 小时内未能达到 ϵ -近似解。CGAL 在 10 个实例中有 7 个失败，而 USBS 即使不用热启动，也能在 28 小时内解出全部实例。

论文：<https://arxiv.org/abs/2312.11801> | 代码：<https://github.com/rangell/usbs>（纯 JAX 实现，支持 CPU / GPU / TPU）

问题出在哪：投影太贵，草图化又挡住了热启动

要理解 USBS 的价值，先看清两代方法各自的瓶颈。经典内点法和一阶方法的核心开销来自“投影到半定锥”这一步：它本质上是对一个 $n \times n$ 矩阵做完整特征分解，代价随 n 立方增长。当 n 到达百万级，这一步就彻底不可行——这也是“SDP 规模上不行”这一印象的技术根源。

草图化方法换了一条路：不再存完整矩阵，而是只维护一个低秩草图，从而把内存和单步开销压下来，规模因此能往前推很多。但它有两个软肋。第一，随着问题增大，收敛所需迭代数也随之上升，慢下来；第二，为保证收敛，它使用随迭代变化的步长 / 惩罚参数表，而热启动想做的正是“跳到一个接近最优的初值再继续”，这与固定的迭代参数表天然冲突。结果就是：草图化方法能扩展，却难以真正吃到热启动的红利。

USBS 的贡献：把谱束方法统一起来，并让热启动真正生效

谱束方法 (spectral bundle method) 本是一个很有吸引力的框架：单步开销低、经验收敛快。但此前的版本要么只支持等式约束、要么只支持不等式约束，也没有能在真正大规模 SDP 上评测的独立实现。USBS 把这几项补齐——它是一个面向一般 SDP（同时含等式与不等式约束）的统一谱束方法，具备三个关键特性：(1) 可靠地利用热启动；(2) 可选地叠加矩阵草图，在保持收敛速度的同时把可扩展性大幅拉高；(3) 带有非渐近 (non-asymptotic) 收敛速率的理论保证。此外，方法通过模型参数让用户在“单步复杂度”和“收敛速度”之间自由权衡，并附带一个纯 JAX 的独立实现，可在 CPU、GPU、TPU 上运行。

方法一览：把对偶 SDP 变成一个可以“便宜地建模”的目标

USBS 从对偶半定规划出发，把它改写成一个无约束的“惩罚对偶”目标：把 $C - \lambda^* y$ 的最大特征值、线性项 $\langle b, y \rangle$ ，以及不等式约束的指示函数合成一个函数 $f(y)$ 。形式上， $f(y) := \alpha \cdot [\lambda_{\max}(C - \lambda^* y)]_+ + \langle b, y \rangle + \mathbb{I}_{\mathcal{B}}(y)$ 。随后用邻近束方法 (proximal bundle method) 来最小化它。

关键在于每一步不直接优化真实目标，而是构造一个便宜的下界模型 $\hat{f}_t$ ：这个模型只活在一个低维子空间里——由当前的最大特征向量 (k_c 个) 加上少量携带历史谱信息的方向 (k_p 个) 张成。USBS 对这个模型做一次邻近步，求解一个小规模的 minimax 问题得到候选迭代点 $\tilde{y}_{t+1} = \operatorname{argmin}_y \hat{f}_t(y) + (p/2) \|y - y_t\|^2$ 。接着做一次“下降检验”：如果真实目标的下降量至少达到模型预测下降量的固定比例 β ，即 $\beta \cdot (f(y_t) - \hat{f}_t(\tilde{y}_{t+1})) \leq f(y_t) - f(\tilde{y}_{t+1})$ ，就接受这一步（下降步）；否则只做一次空步 (null step)，但仍用新信息细化模型。因为全程只存少量特征向量、并可选地只存原始矩阵的一个低秩草图，整个过程才得以扩展到极大规模。

这套结构也解释了 USBS 为什么天生适合热启动：它的“状态”是当前解 y 和一小组谱方向，而不是一张与迭代次数绑死的参数表——所以直接从上一个相近问题的解出发继续优化是自然的。下面这张来自交互式实体消解 (entity resolution) 的图，直观展示了这一点。

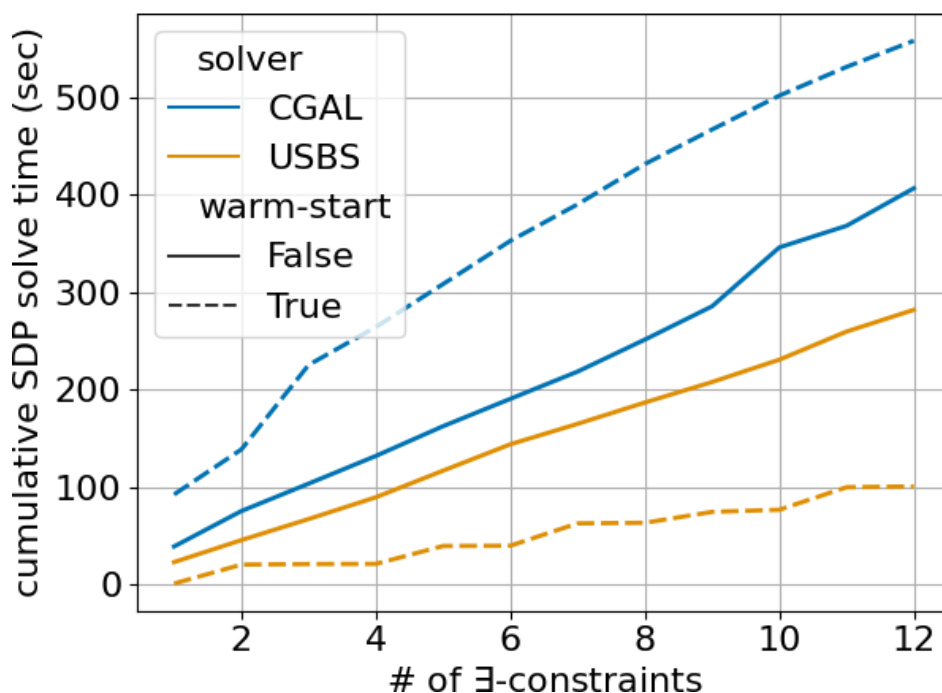


图6 累计 SDP 求解时间（秒，越低越好）随 $\exists$ -约束数量的增长。在作者共指消解任务中，每新增一个 $\exists$ -约束就要解一个与上一个几乎相同的 SDP，因此可以用上一次的解做热启动（虚线）。可以看到 USBs（橙）的热启动明显快于其冷启动，也全面快于 CGAL（蓝），并且规模越大，USBs 相对 CGAL 的优势越明显。

实验：三类问题，一致的结论

论文在三类差异很大的问题上评测 USBs：(1) MaxCut，用十个 DIMACS10 图， n 从 16K 到 3.7M，最大的 333SP 对应超过 10^{13} 个决策变量；(2) 二次分配问题（QAP），用 QAPLIB 与 TSPLIB 的实例，其 SDP 松弛的决策变量数约为 $O(n^4)$ ，在 $n=198$ 时达到约 1.5×10^9 ；(3) 交互式实体消解，用三个作者共指数据集（PubMed、QIAN、SCAD-zbMATH）。三类任务都用“略小一号的相近问题”自然地构造热启动。

MaxCut 的结论就是开篇的图 1：USBs 即使冷启动也能在 28 小时内解出全部 10 个实例，而 CGAL 在 72 小时内有 7 个实例都无法达到目标精度；在超过 2×10^9 个变量的实例上，USBs 比 CGAL 快约 500 $\times$ 。下面两张图转向 QAP，先看单个实例的收敛动态，再看十个实例的整体表现。

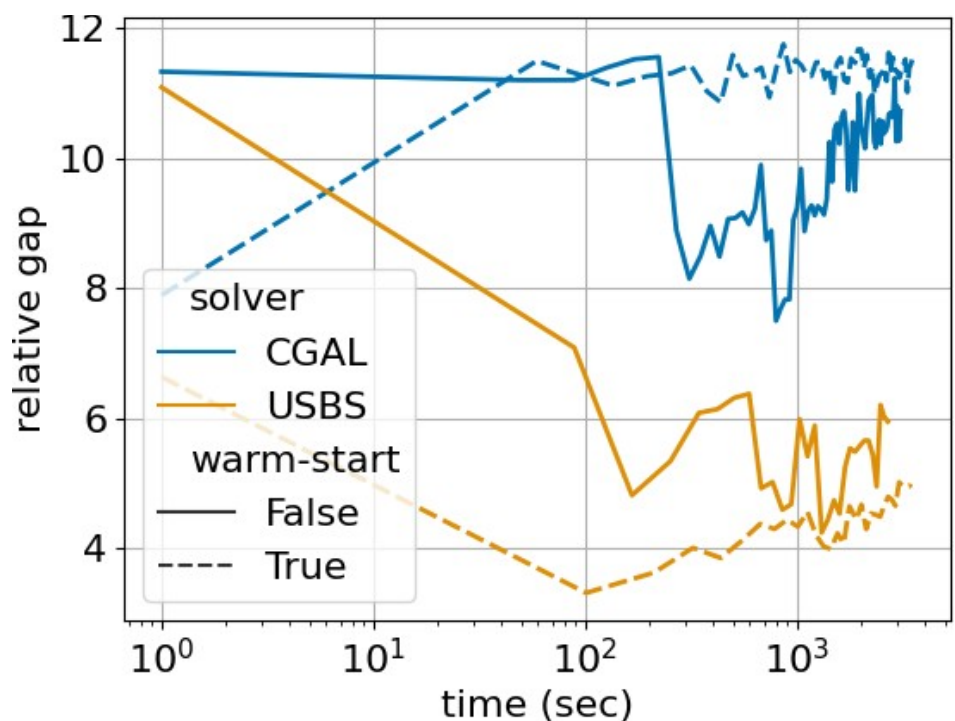


图 2 QAP 实例 pr144 ($n=144$) 在一小时优化内的相对间隙 (relative gap, 越低越好) 随时间变化。实线为冷启动, 虚线为热启动。USBS (橙) 无论冷热都收敛到远低于 CGAL (蓝) 的间隙, 并且能明显借助热启动更快下降; 两种算法都在优化早期就找到了最好的舍入解。

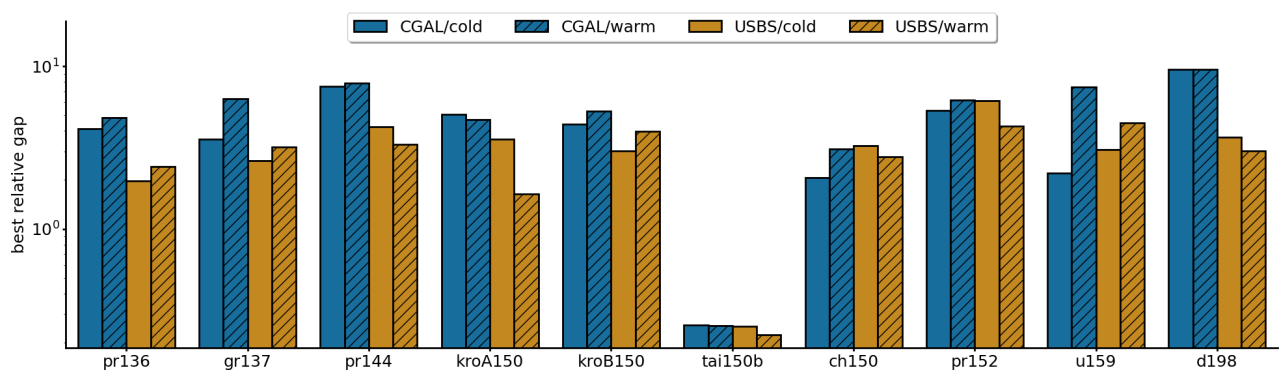


图 5 QAPLIB 与 TSPLIB 十个实例上、一小时内取得的最佳相对间隙 (越低越好)。在绝大多数实例上 USBS (橙) 都优于 CGAL (蓝), 且热启动进一步帮助 USBS 取得更小的间隙; 相比之下, CGAL 明显更难稳定地利用热启动。

把最能说明问题的数字集中列在下表, 便于对照两代求解器的差距。

表 1 USBS 与前一代可扩展求解器 CGAL 的关键对比

指标	CGAL (前 SOTA)	USBS (本文)
MaxCut 求解成功数 (DIMACS10, 10 个)	3 / 10 (72 小时内)	10 / 10 (≤ 28 小时)
$2 \times 10^9+$ 变量 MaxCut 实例上的相对速度	基准 (作为参照)	约 $500\times$ 更快
热启动相对冷启动的收敛加速	常常无法稳定利用	$>100\times$

可扩展规模（决策变量）	—	$>10^{13}$ (333SP, $n \approx 3.7M$)
最大 QAP 松弛规模	—	$n=198$, 约 1.5×10^9 变量

消融：热启动带来 100× 以上加速，kc 比 kp 更关键

两个消融最值得关注。其一是热启动：对 USBS 而言，用上一次的解初始化能把收敛加速 100× 以上，而且重要的是 USBS 真的能吃到这份红利，CGAL 通常做不到。其二是模型参数：当前特征向量的数量 k_c 影响很大，一般越大收敛越好——这与最初的谱束方法（Helmberg-Rendl）把 k_c 固定为 1 形成对比；而携带历史谱信息的方向数 k_p 帮助有限，有时甚至会拖慢收敛。因此论文推荐的默认设置保持 k_c 较大、 k_p 较小（ $r=10$ 、 $\rho=0.01$ 、 $\beta=0.25$ 、 $k_c=10$ 、 $k_p=1$ ）。

小结与边界

USBs 传达的信息很简单：大规模半定规划不必慢、也不必局限于狭窄的一类问题。通过在单一谱束方法里统一等式与不等式约束、用可选草图换取可扩展性、并让热启动真正生效，它把过去被认为“规模上不可行”的 SDP 变成了一个可用工具——尤其适合数据增量到达、或在混合整数 / 交互式循环里反复求解相近问题的流水线。配套的纯 JAX 实现可在 CPU、GPU、TPU 上运行，进一步降低了落地门槛。

当然，边界同样值得记住：这些结论建立在 MaxCut、QAP 与实体消解三类任务上，热启动的收益取决于相邻问题确实“足够相近”；参数 k_c / k_p 的最佳取值也与具体问题和硬件相关。但即便如此，USBs 已经把可求解的 SDP 规模和实用性推进了一大步。

更具体地说，USBs 最能派上用场的场景，是那些需要“反复求解”而非“只解一次”SDP 的流水线：混合整数规划的分支定界、流式或增量到达的数据、以及随着约束不断累积而逐步细化答案的交互式系统。在这些场景里，上一次的求解结果几乎就是下一次的理想初值，而 USBs 正是为“用好它”而设计的。至于那些规模大到其他求解器根本碰不了的问题，可选的草图化则是一个旋钮：用少量精度换取更大的可解规模——这也正是它能在数百万顶点的图上给出结果的原因。

相关资源 —— 论文：<https://arxiv.org/abs/2312.11801> · 代码（纯 JAX，支持 CPU/GPU/TPU）：<https://github.com/rangell/usbs>